

Mia Muzamil Ahmad.
91 Group
Data structure and algorithms

----- 1 lab Parallel Programming -----

```
#include <iostream>
#include <thread>

//We are using the std::thread.
// 17. Дана прямоугольная целочисленная матрица. Распараллеливание по элементам.
// Найти сумму младших разрядов тех чисел, в которых есть повторяющиеся цифры.

const size_t rows = 5;
const size_t columns = 5;
const size_t COUNT = rows * columns;
const size_t NTHREADS = 4;

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;
        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}

int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

// Функция для вычисления локальной суммы в диапазоне индексов
void sum(int (*matrix)[columns], size_t left, size_t right, int& result)
{
    result = 0;
    for (size_t i = left; i < right; ++i) {
        size_t row = i / columns; // Преобразование 1D индекса в строку
        size_t col = i % columns; // Преобразование 1D индекса в столбец
        if (repeat_number(matrix[row][col])) {
            result += least_sig_number(matrix[row][col]);
        }
    }
}

int sum_parallel(int (*matrix)[columns])
{
    std::thread t[NTHREADS - 1];
    int result[NTHREADS - 1]; // Локальные суммы для рабочих потоков
    size_t block = COUNT / NTHREADS; // Размер блока
```

```

    // Запуск рабочих потоков
    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        t[i] = std::thread(sum, matrix, block * i, block * (i + 1),
std::ref(result[i])); // Запуск потока
    }

    // Основной поток обрабатывает последний блок
    int global_sum;
    sum(matrix, block * (NTHREADS - 1), COUNT, global_sum);

    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        t[i].join();
        global_sum += result[i];
    }
    return global_sum;
}

int sum_non_parallel(int (*matrix)[columns])
{
    int result = 0;
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            if (repeat_number(matrix[i][j])) {
                result += least_sig_number(matrix[i][j]);
            }
        }
    }
    return result;
}

void init_matrix_fixed(int (*matrix)[columns])
{
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

void print(int (*matrix)[columns])
{
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';
}

int main()
{

```

```

int matrix[rows][columns];
init_matrix_fixed(matrix);

std::cout << "Матрица:\n";
print(matrix);

std::cout << "Непараллельная сумма: " << sum_non_parallel(matrix) << '\n';
std::cout << "Параллельная сумма: " << sum_parallel(matrix) << '\n';

return 0;
}

```

```

----- 3 lab Parallel Programming -----

const size_t rows = 5;
const size_t columns = 5;
const size_t COUNT = rows * columns;
const size_t NTHREADS = 4;

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;
        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}

int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

// Функция для вычисления суммы в диапазоне индексов
int sum(int (*matrix)[columns], size_t left, size_t right)
{
    int result = 0;
    for (size_t i = left; i < right; ++i) {
        size_t row = i / columns;
        size_t col = i % columns;
        if (repeat_number(matrix[row][col])) {
            result += least_sig_number(matrix[row][col]);
        }
    }
    return result;
}

int sum_parallel(int (*matrix)[columns])
{
    std::future<int> t[NTHREADS - 1]; // Массив асинхронных задач
    size_t block = COUNT / NTHREADS; // Размер блока

    // Запуск асинхронных задач

```

```

    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        t[i] = std::async(std::launch::async, sum, matrix, block * i, block * (i +
1));
    }

    // Основной поток обрабатывает последний блок
    int global_sum = sum(matrix, block * (NTHREADS - 1), COUNT);

    // Сбор результатов асинхронных задач
    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        global_sum += t[i].get(); // Получение результата
    }
    return global_sum;
}

int sum_non_parallel(int (*matrix)[columns])
{
    return sum(matrix, 0, COUNT);
}

void init_matrix_fixed(int (*matrix)[columns])
{
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

void print(int (*matrix)[columns])
{
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';
}

int main()
{
    int matrix[rows][columns];
    init_matrix_fixed(matrix);

    std::cout << "Матрица:\n";
    print(matrix);

    std::cout << "Непараллельная сумма: " << sum_non_parallel(matrix) << '\n';
    std::cout << "Параллельная сумма: " << sum_parallel(matrix) << '\n';

    return 0;
}

```

```

----- 4 lab Parallel Programming -----
const size_t rows = 5;
const size_t columns = 5;
const size_t COUNT = rows * columns;
const size_t NTHREADS = 4;

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;
        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}

int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

// Функция для вычисления локальной суммы в диапазоне индексов
void sum(int (*matrix)[columns], size_t left, size_t right, std::atomic_int& result)
{
    int local_res = 0;
    for (size_t i = left; i < right; ++i) {
        size_t row = i / columns;
        size_t col = i % columns;
        if (repeat_number(matrix[row][col])) {
            local_res += least_sig_number(matrix[row][col]);
        }
    }
    result += local_res; // Атомарное обновление общей суммы
}

int sum_parallel(int (*matrix)[columns])
{
    std::thread t[NTHREADS - 1]; // Массив потоков
    size_t block = COUNT / NTHREADS;
    std::atomic_int global_sum{ 0 }; // Атомарная общая сумма

    // Запуск рабочих потоков
    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        t[i] = std::thread(sum, matrix, block * i, block * (i + 1),
std::ref(global_sum));
    }

    // main thread
    sum(matrix, block * (NTHREADS - 1), COUNT, global_sum);

    // waiting
    for (size_t i = 0; i < NTHREADS - 1; ++i) {
        t[i].join();
    }
}

```

```

        return global_sum;
    }

int sum_non_parallel(int (*matrix)[columns])
{
    std::atomic_int result{ 0 };
    sum(matrix, 0, COUNT, result);
    return result;
}

void init_matrix_fixed(int (*matrix)[columns])
{
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

void print(int (*matrix)[columns])
{
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';
}

int main()
{
    int matrix[rows][columns];
    init_matrix_fixed(matrix);

    std::cout << "Матрица:\n";
    print(matrix);

    std::cout << "Непараллельная сумма: " << sum_non_parallel(matrix) << '\n';
    std::cout << "Параллельная сумма: " << sum_parallel(matrix) << '\n';

    return 0;
}

```

```

----- 5 lab Parallel Programming -----

const int COUNT = 25;
const int THREAD = 4;
const size_t rows = 5;
const size_t columns = 5;

struct Pair {
    size_t left, right;
};

```

```

// Класс потокобезопасной очереди
class ThreadSafeQueue {
private:
    std::mutex mutex; // Мьютекс для потокобезопасности
    std::queue<Pair> queue;
public:
    ThreadSafeQueue() {}
    void push(Pair elem) {
        queue.push(elem);
    }
    bool try_pop(Pair& elem) {
        bool result = false;
        if (!queue.empty()) {
            std::lock_guard<std::mutex> lock(mutex);
            if (!queue.empty()) {
                result = true;
                elem = queue.front();
                queue.pop();
            }
        }
        return result;
    }
    bool empty() {
        std::lock_guard<std::mutex> lock(mutex);
        return queue.empty();
    }
};

ThreadSafeQueue TSQ; // Глобальная очередь
std::mutex mutex_task; // Мьютекс для защиты общей суммы

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;
        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}

int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

// Функция потока для обработки диапазона элементов
void process_elements(int (*matrix)[columns], int& global_sum)
{
    Pair pair;
    while (TSQ.try_pop(pair))
    { // Попытка извлечь диапазон из очереди
        long local_sum = 0;
        for (size_t i = pair.left; i < pair.right; ++i) {

```

```

        size_t row = i / columns;
        size_t col = i % columns;
        if (repeat_number(matrix[row][col])) {
            local_sum += least_sig_number(matrix[row][col]);
        }
        //std::this_thread::sleep_for(std::chrono::milliseconds(1)); // Имитация
работы
    }
    std::lock_guard<std::mutex> locker(mutex_task); // Блокировка мьютекса
    global_sum += local_sum;
}
}

```

```

int sum_parallel(int (*matrix)[columns])
{
    size_t chunk = 10; // Размер блока
    size_t left = 0, right = 0;
    while (right != COUNT)
    { // Разделение на блоки
        left = right;
        right = right + chunk < COUNT ? right + chunk : COUNT;
        TSQ.push(Pair{ left, right }); // Добавление диапазона в очередь
    }

    std::thread thr[THREAD - 1]; // Массив потоков
    int global_sum = 0; // Общая сумма
    for (size_t i = 0; i < THREAD - 1; ++i) {
        thr[i] = std::thread(process_elements, matrix, std::ref(global_sum));
    }

    //main thread
    process_elements(matrix, global_sum);

    for (size_t i = 0; i < THREAD - 1; ++i) {
        thr[i].join();
    }

    return global_sum;
}

int sum_non_parallel(int (*matrix)[columns]) {
    int result = 0;
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            if (repeat_number(matrix[i][j])) {
                result += least_sig_number(matrix[i][j]); // Добавляем младшую цифру
            }
        }
    }
    return result;
}

void init_matrix_fixed(int (*matrix)[columns]) {
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
    };
}

```



```

        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

void print(int (*matrix)[columns]) {
    for (size_t i = 0; i < rows * columns; ++i) {
        size_t row = i / columns;
        size_t col = i % columns;
        std::cout << matrix[row][col] << ' ';
    }
    std::cout << '\n';
}

int main()
{
    int matrix[rows][columns];
    init_matrix_fixed(matrix); // Используем фиксированную матрицу для суммы 62

    std::cout << "Array: ";
    print(matrix);

    std::cout << "Non parallel sum: " << sum_non_parallel(matrix) << '\n';
    std::cout << "Parallel sum:      " << sum_parallel(matrix) << '\n';

    return 0;
}

```

----- 6 lab Parallel Programming -----

```

#include "queue.h"
#include <iostream>
#include <condition_variable>
#include <thread>
#include <mutex>
#include <Windows.h>

const size_t rows = 5;
const size_t columns = 5;
const size_t COUNT = rows * columns;
const size_t NTHREADS = 5; // 2 производителя + 3 потребителя

ThreadsafeQueue que; // Потокбезопасная очередь
std::condition_variable cv; // Условная переменная
std::mutex mut; // Мьютекс для cv и global_sum
volatile long volume_work_producer = 5; // Количество задач для производителей (5
строк)
volatile long volume_work_consumer = 5; // Количество задач для потребителей
std::mutex sum_mutex; // Мьютекс для global_sum
long global_sum = 0; // Общая сумма

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;

```

```

        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}
int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

void init_matrix_fixed(int (*matrix)[columns])
{
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

int sum_non_parallel(int (*matrix)[columns])
{
    int result = 0;
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            if (repeat_number(matrix[i][j])) {
                result += least_sig_number(matrix[i][j]);
            }
        }
    }
    return result;
}

// Функция производителя
void task_producer(int id, int (*matrix)[columns])
{
    srand(GetTickCount64() + id); // Уникальный seed для каждого производителя

    //checks if tasks remain.
    while (_InterlockedExchangeAdd(&volume_work_producer, -1) > 0)
    {
        size_t row = volume_work_producer;
        if (row < rows) {
            int* arr = new int[columns]; // Массив для строки
            for (size_t j = 0; j < columns; ++j) {
                arr[j] = matrix[row][j]; // Копируем строку
            }
            que.push(arr); // Добавляем в queue
            {
                std::lock_guard<std::mutex> lock(mut);
                cv.notify_all(); // Уведомляем потребителей
            }
        }
    }
}

```

```

        //std::this_thread::sleep_for(std::chrono::milliseconds(2));
    }
}

void task_consumer(int (*matrix)[columns])
{
    while (_InterlockedExchangeAdd(&volume_work_consumer, -1) > 0)
    {
        int* arr = nullptr;
        {
            std::unique_lock<std::mutex> locker(mut);
            cv.wait_for(locker, std::chrono::seconds(5), []() { return !que.empty();
});
        }
        if (que.try_pop(arr)) // Извлечение строки
        {
            int local_sum = 0;
            for (size_t j = 0; j < columns; ++j) {
                if (repeat_number(arr[j])) {
                    local_sum += least_sig_number(arr[j]); // Локальная сумма
                }
            }
            {
                std::lock_guard<std::mutex> lock(sum_mutex); // Защита global_sum
                global_sum += local_sum; // Обновление общей суммы
            }
            delete[] arr; // Освобождение памяти
            //std::this_thread::sleep_for(std::chrono::milliseconds(5));
        }
        else
        {
            _InterlockedExchangeAdd(&volume_work_consumer, 1); // Возврат задачи
        }
    }
}

int sum_parallel(int (*matrix)[columns])
{
    global_sum = 0;
    std::thread wor_thread[NTHREADS];

    // Запуск производителей и потребителей
    for (size_t i = 0; i < NTHREADS; ++i) {
        if (i < 2)
            wor_thread[i] = std::thread(task_producer, 10 * i, matrix); //
        Производители
        else
            wor_thread[i] = std::thread(task_consumer, matrix); // Потребители
    }

    for (size_t i = 0; i < NTHREADS; ++i) {
        wor_thread[i].join();
    }

    return global_sum;
}

```

```

void print(int (*matrix)[columns])
{
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';
}

int main()
{
    int matrix[rows][columns];
    init_matrix_fixed(matrix);

    std::cout << "Матрица:\n";
    print(matrix);

    std::cout << "Непараллельная сумма: " << sum_non_parallel(matrix) << '\n';
    std::cout << "Параллельная сумма: " << sum_parallel(matrix) << '\n';

    return 0;
}

```

----- 7 lab Parallel Programming -----

```

#include <iostream>
#include <omp.h>

const size_t rows = 5;
const size_t columns = 5;
const size_t COUNT = rows * columns;
const size_t NTHREADS = 4;

bool repeat_number(int num)
{
    num = std::abs(num);
    bool digit_seen[10] = { false };
    bool flag = false;
    while (num > 0 && !flag) {
        int digit = num % 10;
        if (digit_seen[digit])
            flag = true;
        digit_seen[digit] = true;
        num /= 10;
    }
    return flag;
}

int least_sig_number(int num)
{
    return std::abs(num) % 10;
}

int sum(int (*matrix)[columns], size_t size)
{
    int result = 0;
    for (size_t i = 0; i < size; ++i) {

```

```

        size_t row = i / columns;
        size_t col = i % columns;
        if (repeat_number(matrix[row][col])) {
            result += least_sig_number(matrix[row][col]);
        }
    }
    return result;
}

int sum_parallel(int (*matrix)[columns])
{
    int global_sum = 0;
    omp_set_num_threads(NTHREADS); // Установка числа потоков
    #pragma omp parallel shared(matrix) reduction(+:global_sum)
    {
        int local_sum = 0;
        #pragma omp for //divide the loop
        for (int i = 0; i < COUNT; ++i) {
            size_t row = i / columns;
            size_t col = i % columns;
            if (repeat_number(matrix[row][col])) {
                local_sum += least_sig_number(matrix[row][col]);
            }
        }
        global_sum += local_sum;
    }
    return global_sum;
}

void init_matrix_fixed(int (*matrix)[columns])
{
    int fixed_matrix[rows][columns] = {
        {99, 99, 99, 99, 11},
        {99, 12, 23, 45, 67},
        {22, 89, 78, 56, 34},
        {55, 23, 45, 67, 89},
        {78, 56, 34, 12, 23}
    };
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < columns; ++j)
            matrix[i][j] = fixed_matrix[i][j];
}

int sum_non_parallel(int (*matrix)[columns])
{
    return sum(matrix, COUNT);
}

void print(int (*matrix)[columns])
{
    for (size_t i = 0; i < rows; ++i) {
        for (size_t j = 0; j < columns; ++j) {
            std::cout << matrix[i][j] << " ";
        }
        std::cout << '\n';
    }
    std::cout << '\n';
}

```

```
int main()
{
    int matrix[rows][columns];
    init_matrix_fixed(matrix);

    std::cout << "Матрица:\n";
    print(matrix);

    std::cout << "Непараллельная сумма: " << sum_non_parallel(matrix) << '\n';
    std::cout << "Параллельная сумма: " << sum_parallel(matrix) << '\n';

    return 0;
}
```